Ders 13 Genişletilmiş Notu: Özellik Seçimi, Filtre ve Sarmalayıcı Yöntemler ve Temel Bileşen Analizi

Ders 13 Genişletilmiş Notu: Özellik Seçimi, Filtre ve Sarmalayıcı Yöntemler ve Temel Bileşen Analizi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Veri Kümesi Hazırlığı ve Özellikler
- 2. İyi Bir Özelliğin Ölçütü
- 3. Kategorik ve Sayısal Özellikler
- 4. Kategorik Özellikleri Sayıya Çevirme
- 5. Özellik Sayısının Etkisi
- 6. Özellik Azaltma: Seçim ve Çıkarım
- 7. Özellik Seçiminde Üç Yaklaşım ve Filtre Yöntemleri
- 8. İstatistik Hatırlatması: Ortalama, Varyans ve Kovaryans
- 9. Pearson Korelasyon Katsayısı
- 10. Ki-Kare Testi
- 11. Bilgi Kazancı ve Filtre Yöntemleriyle Adım Adım Eleme
- 12. Sarmalayıcı Yöntemler: İleri Seçim ve Geri Eleme
- 13. Plus-L Minus-R Seçimi ve Çift Yönlü Arama
- 14. Filtre Yöntemleri Uygulamada Nasıl Kullanılır
- 15. Özellik Çıkarımı, Özdeğer ve Özvektör
- 16. PCA'nın Sezgisi ve Özyüzler
- 17. PCA ile Yüz Tanıma: Eğitim ve Test Adımları
- 18. Örnek: 3×3 Görüntülerle PCA
- 19. Kaç Özvektör Seçilmeli
- 20. PCA'nın Kullanım Alanları
- 21. Otokodlayıcı ve Derin Öğrenmede Özellik Seçimi
- 22. Gömülü Yöntemler

1. Veri Kümesi Hazırlığı ve Özellikler

Bu derste konu özellik seçimidir (feature selection): bir öğrenme modeli için hangi özelliklerin kullanılması gerektiği ve bunun nasıl belirleneceği.

Hoca konuya bir öğrenme modelinin nasıl hazırlandığını hatırlatarak başladı. İlk iş bir veri kümesi hazırlamaktır, yani veri toplamaktır. Veri toplanırken o veriyle ilgili önemli bütün bilgiler elde edilmeye çalışılır. Örneğin yüz tanıma yapılacaksa kişinin farklı açılardan çekilmiş fotoğrafları kullanılır. Peki bu fotoğraflardan hangi bilgiler özellik olarak alınabilir? Burnun büyüklüğü, iki göz arasındaki mesafe gibi bilgiler. İlk yüz tanıma çalışmaları tam olarak böyleydi: yüz üzerinde üçgenler tanımlanır, bu üçgenlerin kenar mesafelerine göre tanıma yapılırdı. Bunlar, tanıma için faydalı olacağı düşünülerek elle seçilmiş özelliklerdir.

Yani veri kümesi hazırlanırken önce elle (manual) bir iş yapılır. Örnekler toplanır ve bu örnekler, bizim için önemli olduğunu düşündüğümüz özellikleriyle birlikte toplanır.

Hocanın ikinci örneği bir bankadır. Müşterileri bankadaki hesaplarını kullanma alışkanlıklarına göre gruplamak istediğinizi düşünün. Bunun için "bu müşterinin bütün işlemlerini (transaction) toplayayım ve kullanayım" dersiniz. Peki bu ne kadar doğrudur? Hissiyatla seçilen bu özellikler gerçekten kullanılması gereken özellikler midir? Dersin asıl sorusu budur.

2. İyi Bir Özelliğin Ölçütü

Hoca, şimdiye kadar görülen bütün yöntemlerin ortak hedefini hatırlattı. Makine öğrenmesiyle tanıma, sınıflandırma ya da öbekleme (clustering) yaparken hedef, aynı sınıftaki örnekleri olabildiğince bir arada, farklı sınıftaki örnekleri olabildiğince uzak tutmaktır. Başka bir deyişle sınıf içi benzerliğin çok, sınıflar arası benzerliğin az olduğu bir sistem kurulmaya çalışılır.

> **Hocanın vurgusu:** Yaptığımız bütün makine öğrenmesi modellerinde ana hedefimiz budur: sınıf içi benzerlik çok, sınıflar arası benzerlik az olmalıdır.

Bu hedefe ulaşmada en önemli noktalardan biri, veri kümesini oluşturan örneklerin özellikleridir. Hoca bunu tek boyutlu bir çizimle anlattı. İki sınıfa ait örneklerin her birinin tek bir özelliği olsun; tek özellik olduğu için örnekler tek boyutlu bir eksen üzerinde gösterilebilir. Bir durumda iki sınıfın örnekleri eksen üzerinde iç içe geçmiştir. Diğer durumda A sınıfının örnekleri bir tarafta, B sınıfının örnekleri öbür tarafta toplanmıştır. Tercih edilecek özellik ikincisidir, çünkü bu özelliğe göre A ve B sınıfları birbirinden net olarak ayrılmaktadır.

Hoca bir de ayırt edici olmayan özelliğe örnek verdi. Yüz tanıma yapıyorsanız göz rengini kullanmamanız gerekir. Göz rengi bazı durumlarda ayırt edici gibi görünebilir, ama bir kişiyi tanımak için ayırt edici bir özellik değildir; farklı insanlarda ortak bulunabilir.

Buradan özellik seçimindeki temel ilke çıkar: Önce özellikler toplanır, ama sonra bunların içinden ayırt edici, karakteristik olanlar ayıklanmalıdır.

Bu ayıklama nasıl yapılabilir? Birinci yol elle, yani hissiyatla yapmaktır. Hoca, makine öğrenmesinde bu işin çok uzun yıllar böyle yapıldığını ve bunun pek doğru olmadığını söyledi. Elbette öncesinde bir gözlem vardı, ama özellikler matematiksel temellere dayanmadan "şu özellikleri seçtim" denilerek seçiliyor ve işlemler bunlar üzerinden yapılıyordu. Görünüşte özellikler arasında bir bağlantı yoktu, ama bunun sağlaması matematiksel olarak yapılmadığı için sorun çıkıyordu. Özellikler matematiksel bir değerlendirmeyle seçilseydi belki daha başarılı modeller elde edilecekti.

> **Hocanın vurgusu:** Özellikleri hissiyatla seçmek uzun yıllar yapıldı, ama doğru değil. Bugün matematiksel temelli özellik seçme yöntemleri yaygın ve başarılı biçimde kullanılıyor.

Daha sonra hoca, karakteristik bir özelliğin taşıması gereken iki şartı açıkça sıraladı:

- **Sınıflar arası farkı ve sınıf içi benzerliği artırmalı.** Her iki sınıfta da ortak geçen, bu yüzden ayırt ediciliği olmayan özellikler kullanılmamalıdır.
- **O sınıf için gerçekten ayırt edici olmalı.** A ve B diye iki sınıf olsun. Bir özellik A sınıfında daha çok geçiyor olabilir, öbür sınıfla da ilgisi olmayabilir. Ama baktığınızda ayırt edicilik açısından önemli bir bilgi sağlamıyorsa o sınıf için de iyi bir özellik değildir.

Yani bir özelliğin hem sınıf içinde iyi olması hem de sınıflar arasında ayırt ediciliği sağlaması gerekir.

3. Kategorik ve Sayısal Özellikler

Özellikler değerlendirilirken iki tür özellik vardır: kategorik özellikler (categorical features) ve sayısal özellikler (numerical features).

Kategorik özellik. Bir şekilden bahsediyorsanız "yuvarlak, elips, kare" dersiniz; bu kategorik bir özelliktir. Bir kişinin yaşadığı şehirden bahsediyorsanız "İstanbul, Ankara, İzmir" dersiniz; bu da kategoriktir. Kategorik özellik, değerleri arasında ölçüm yapılamayan özelliktir. "İstanbul ile Ankara arasındaki mesafe 3'tür" diyemezsiniz. İki şehir arasında elbette bir kilometre mesafesi vardır, ama bu İstanbul ile Ankara'nın birbirine benzerliğini gösteren bir şey değildir. Aynı şekilde "kare daireye ne kadar benziyor" gibi bir karşılaştırma yapmak da mümkün değildir.

Sayısal özellik. İki kişinin kilosunu karşılaştırdığınızı düşünün: biri 45 kilo, öbürü 57 kilo. Bunlar ölçülebilir değerlerdir; aralarında mesafe hesaplanabilir, benzerlik değerlendirilebilir. Kişinin yaşı, kilosu, bir evin yüzölçümü, müşterinin yaptığı harcama miktarı sayısal özelliklerdir. Bunlar üzerinde matematiksel işlem ve karşılaştırma yapılabilir.

Kategorik özellikler modellerde nasıl kullanılır? Önceki derslerde görüldüğü gibi bazı yöntemler kategorik özellikleri doğrudan kullanabilir:

- **Karar ağacı:** Kategorik özelliği doğrudan kullanır. Çünkü orada istatistik girer işin içine; örneğin "İstanbul'da oturan kaç kişi var" sorusuyla sayım yapılır ve kategorik özellik kullanılabilir hale gelir.
- **Naive Bayes:** Yine aynı mantıkla kategorik özelliği kullanabilir. Naive Bayes ile yapılan doküman sınıflandırma örneğinde kelimeler kullanılmıştı. Kelime kategorik bir özelliktir, ama kelimelerin kullanım sıklıklarına (frekanslarına) bakıldığı için kullanılabilir hale gelir. Yani özelliğin kendisi değil, yöntemin frekansa dayalı olması bunu sağlar.

4. Kategorik Özellikleri Sayıya Çevirme

Kategorik özelliklerle yapılabilecek ikinci şey, onları sayısal özelliğe dönüştürmektir. Hoca bunun son yıllarda çok popüler olduğunu, ama avantaj ve dezavantajlarının tartışmalı olduğunu söyledi. İki yol vardır: tamsayı kodlama (integer encoding) ve tek-sıcak kodlama (one-hot encoding).

4.1 Tamsayı Kodlama (Integer Encoding)

Meyvelerin renklerinin özellik olarak saklandığını düşünün. Bu renk bir görüntü bilgisinden ölçülmüş değil, düz bir etiket olarak saklanmış olsun: red, green, blue. Tamsayı kodlamada her değere bir sayı verilir:

Renk	Kod
red	1
green	2
blue	3

Böylece özellik sayısal hale gelir. Bu, matematiksel işlem gerektiren makine öğrenmesi algoritmalarında da artık bu özelliğin kullanılabileceği anlamına gelir. Avantajı budur.

Ama bu kodlama ölçme için iyi değildir. Kırmızının yeşile mesafesi 1'dir gibi bir şey yoktur. Sadece sayısal hale dönüştürülmüştür. Mesafe ölçmeye kalkarsanız, örneğin Öklid mesafesi (Euclidean distance) hesaplarsanız, bu tehlikelidir; "mavi kırmızıya 2 benziyor" gibi anlamsız bir sonuç çıkar. Bu yüzden nerede kullanılacağına dikkat etmek gerekir.

4.2 Tek-Sıcak Kodlama (One-Hot Encoding)

Hocanın daha çok kullanıldığını söylediği ikinci yaklaşım one-hot kodlamadır. Özelliğin alabileceği üç değer (red, green, blue) varsa üç ayrı özellik eklenir: red bir özellik, green bir özellik, blue bir özellik olur. Örneğin kırmızı bir elmada red 1, green ve blue 0 yapılır. Yeşil bir ıspanakta green 1, red ve blue 0 yapılır.

Örnek	red	green	blue
kırmızı elma	1	0	0
yeşil ıspanak	0	1	0

Bu gösterimle Hamming mesafesi (Hamming distance) kullanılabilir: aynı olanların mesafesi 0, farklı olanların mesafesi 1 mantığıyla çalışılır. Normal mesafe hesapları da yapılabilir. Biri kırmızı, öbürü yeşil iki örneğin mesafesine bakılırsa red sütunları arasındaki fark 1, green sütunları arasındaki fark da 1 olur. Böylece iki örneğin farklılığı ortaya çıkarılmış olur.

Soru: Hoca sınıfa one-hot kodlamanın sakıncasının ne olabileceğini sordu. **Cevap:** Bir öğrenci "içinde çok fazla sıfır olacak" dedi. Hoca bunu doğru buldu ve şöyle tamamladı: Çok fazla sıfırdan da öte, çok fazla özellik oluşur.

5. Özellik Sayısının Etkisi

Hoca one-hot kodlamanın sakıncasını kendi kredi kartı sahtekarlığı (fraud) projesinden bir örnekle anlattı. Banka müşterisinin bir işleminde "hangi ülkede yapıldı" bilgisi varsa ve 80 farklı ülke varsa, one-hot ile 80 özellik eklenir. "Hangi şehirde yapıldı", "hangi semtte yapıldı" gibi bilgiler eklendikçe bunun sonu gelmez. Her şehir bir özellik olur: olay o şehirde olmuşsa o şehrin değeri 1, diğer bütün şehirlerin değeri 0 olur.

Hocanın ekibi bu projede müşterinin işlemleriyle ilgili yaklaşık 40 özellikle başlıyordu. One-hot kodlamayla şehirler ve benzeri bilgiler dönüştürülünce yaklaşık 4000 özelliğe çıkıyorlardı.

> **Hocanın vurgusu:** One-hot kodlamanın sakıncası iki yönlüdür. Bir: çoğunluğu sıfır olan bir özellik kümesi oluşur. İki: özellik sayısı bu kadar artması iyi bir şey değildir.

Hoca burada bir dengeye dikkat çekti. Çok az özellikle çalışmak da iyi değildir. Örneğin kredi kartı sahtekarlığını yalnız harcama miktarına ve harcamanın saatine bakarak tespit etmek kolay değildir. Ama alışverişin hangi şehirde yapıldığı, hangi firmada yapıldığı, ne kadar harcandığı, kaç kez yapıldığı gibi bir sürü özellik eklendiğinde olay daha anlaşılır hale gelir. Yani özelliklerin az olmaması iyidir.

Fakat bu şekilde türetilmiş özelliklerle özellik sayısının binlere, üç binlere, beş binlere çıkması iyi değildir:

- **Hız açısından:** Özellik sayısı arttıkça sistem bunlar üzerinde iş yaparken yavaşlar.
- **Başarı açısından:** Modelin başarısını olumsuz etkileyebilir. Bu yüzden özelliklerin yeniden değerlendirilmesi gerekir.

6. Özellik Azaltma: Seçim ve Çıkarım

Karakteristik özellikleri bulmak için yapılacak ilk iş özellikleri azaltmaktır. Azaltmaktan kastedilen, eldeki ham özelliklerden anlamlı ve karakteristik olanları çıkarmaktır. Bunun için iki temel yaklaşım vardır: özellik seçimi (feature selection) ve özellik çıkarımı (feature extraction).

Özellik seçimi. Elinizde N özellikten oluşan bir x örneği olsun. Özellik seçiminde bunların içinden M tanesi seçilir; burada M, N'den (genellikle çok) küçüktür: $M \ll N$. Seçilenler ayırt edici özelliklerdir. Burada bir dönüşüm yoktur. Örneğin "boyu seçtim, kiloyu seçtim, yaşı seçtim" denir; özellikler olduğu gibi kalır, yalnız bir kısmı seçilir.

Özellik çıkarımı. Yine elinizde N özellik vardır. Ama çıkarım yöntemleri kullanıldığında bu özelliklerden farklı, yeni M özellik oluşturulur. Yeni özelliklerin baştaki özelliklerle doğrudan bir ilgisi yoktur. Boy, yaş, kilo diye verilen bilgiler öbür tarafta farklı bilgiler olarak yer alır. Burada da hedef aynıdır: yeni özellikler baştakilerden daha az sayıda ve daha ayırt edici olmalıdır.

Hoca derse özellik seçimiyle, yani özellikleri dönüştürmeden sadece eleyerek kullanmakla başladı.

7. Özellik Seçiminde Üç Yaklaşım ve Filtre Yöntemleri

Özellik seçiminde üç temel yaklaşım vardır:

- **Filtre (filter) yaklaşımı**
- **Sarmalayıcı (wrapper) yaklaşımı**
- **Gömülü (embedded) yaklaşım**

Filtre yaklaşımında özellikler istatistiksel olarak değerlendirilir ve model için ayırt edicilikleri belirlenmeye çalışılır. Hoca filtre yöntemlerinin akışını şöyle anlattı: Toplanan örneklere ait bütün özellikler elinizdedir. Filtre yöntemi bunların içinden bir alt küme seçer; yani bir kısım özellik, gereksiz oldukları gösterilerek elenir. Seçilen alt küme öğrenme algoritmasına verilir.

```
tüm özellikler --> alt küme seçimi (istatistik) --> öğrenme algoritması --> performans
```

Ayırt edicilikten bahsediliyorsa ilk akla gelen, özellikler arasında korelasyon olup olmadığına bakmaktır. Hoca korelasyonun en çok kullanılan filtre yöntemlerinden biri olduğunu söyledi. Ama korelasyona girmeden önce bazı temel istatistik kavramlarını hatırlattı.

> **Hocanın vurgusu:** Kovaryans ve korelasyon, doktora yeterlik sınavına girenlere en çok sorulan sorulardan biridir. Bu soru makine öğrenmesi kısmından değil, istatistik kısmından gelir.

8. İstatistik Hatırlatması: Ortalama, Varyans ve Kovaryans

Ortalama (beklenen değer). N örneğiniz olsun ve bu örneklerin dağılımı hakkında fikir edinmek isteyin. İlk yapılabilecek şey ortalamayı almaktır. Ortalama aslında beklenen değerdir (expected value):

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$$

Ortalama size örnekler hakkında bir bilgi verir. Örneğin "İstanbul'da ocak ayındaki ortalama hava sıcaklığı 12 derecedir" ya da "makine öğrenmesi dersinde sınıfın ortalaması 55'tir" demek fena bir bilgi değildir.

Ama ortalama dağılım hakkında bilgi vermez. Hoca bunu iki örnek grubuyla gösterdi: A sınıfının ve B sınıfının örneklerinin ortalaması aynı (3) çıkıyordu, ama iki grup birbirine hiç benzemiyordu. Demek ki ortalama tek başına yetmez, başka bir ölçüye ihtiyaç vardır.

Varyans. İkinci ölçü varyanstır. Varyans, örneklerin ortalamadan ne kadar uzaklaştığını, yani standarttan ne kadar sapıldığını gösterir. Varyans, standart sapmanın karesidir:

$$\sigma^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \mu)^2$$

Hoca bölme konusunu şöyle açıkladı: İnceleme bütün veri (popülasyon) üzerinde yapılıyorsa N'e, temsili bir grup (örneklem) üzerinde yapılıyorsa $N-1$'e bölünür. Varyansta her örneğin ortalamadan ne kadar uzakta olduğu ölçülür. Böylece A ve B gruplarının dağılımlarının farklı olduğu, ortalamadan anlaşılamazken, varyanstan anlaşılır.

Kovaryans. Asıl merak ettiğimiz şey, elimizdeki özelliklerin birbirlerine göre durumudur. Bu, bir özelliğin sınıfa göre durumu da olabilir, iki özelliğin birbirine göre durumu da. İki özelliğin birlikte değişimi görülmek isteniyorsa kovaryansa bakılır. Hocanın örneği: İlkokul çağındaki çocuklarda boy uzadıkça genellikle kilo da artar. Çocuklar gelişme çağındadır ve bu iki değişken arasında birlikte bir değişme vardır.

Varyans bir değişkenin kendi değişimiydi; kovaryans iki değişkenin birlikte değişimidir:

$$\text{cov}(X, Y) = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \mu_x)(y_i - \mu_y)$$

Hoca bölme için yine N ya da $N-1$'in kullanılabileceğini söyledi. Bu hesapla X ve Y'nin birbirlerine göre değişimi görülür.

9. Pearson Korelasyon Katsayısı

Kovaryansın üzerine kurulan korelasyon katsayısı, iki özelliğin ya da bir özellik ile sınıfın birbirleriyle ilişkili (korele) olup olmadığını gösterir. Bunun için en çok kullanılan yöntemlerden biri Pearson korelasyon katsayısıdır (Pearson Correlation Coefficient), kısaca r.

Pearson korelasyon katsayısı iki değişken arasındaki doğrusal (lineer) ilişkiyi gösterir ve -1 ile $+1$ arasında değişir:

- $r > 0$**, pozitif korelasyon:** Biri artarken öbürü de artar. Ör. çocukların boyu artarken kilolarının da artması.
- $r < 0$**, negatif korelasyon:** Biri artarken öbürü azalır. Ör. çok sigara içildikçe ömrün kısalması.
- $r \approx 0$**, ilişki yok:** Ör. yağmur yağan günler ile televizyonda belli bir filmin (hocanın örneğinde Pembe Panter) oynaması. İkisi arasında hiçbir ilgi yoktur; bunlar korele olmayan bilgilerdir.

  Ek bilgi (derste anlatılmadı): Pearson katsayısı, kovaryansın iki değişkenin standart sapmalarına bölünmesiyle bulunur: $r = \dfrac{\mathrm{cov}(X,Y)}{\sigma_X \sigma_Y}$. Bu bölme kovaryansı birimden bağımsız hale getirir ve değeri $[-1, 1]$ aralığına sıkıştırır. r yalnız doğrusal ilişkiyi ölçer; $y = x^2$ gibi doğrusal olmayan güçlü bir ilişkide bile r sıfıra yakın çıkabilir.

Hoca bu üç durumu bir x-y grafiği üzerinde çizdi:

- Noktalar artan bir doğru etrafında toplanıyorsa iki özellik birbiriyle çok korelidir (ör. boy arttıkça kilonun da çok artması).
- Noktalar hiçbir doğruya oturmuyorsa doğrusal ilişki yoktur, korelasyon sıfırdır. Hoca, "x üç iken y yedi, x dört iken y on iki" gibi rastgele noktalar koyarak bunu göstermeye çalıştı.
- Noktalar azalan bir doğru etrafında toplanıyorsa negatif korelasyon vardır.

Filtre yöntemi olarak kullanımı. Hoca korelasyonun özellik seçiminde iki şekilde kullanıldığını anlattı:

- **İki özellik arasında:** İki özellik arasında korelasyon yüksekse iki özelliği birden kullanmak gerekmez. Birini kullanmak yeterlidir. Genellikle özelliklerin birbirlerine göre durumları değerlendirilir; korelasyonu yüksek olan özelliklerden birinin tercih edilmesi uygundur.
- **Özellik ile sınıf arasında:** Hoca önce "iki farklı sınıftaki iki özellik arasında" diye başladı, sonra kendini düzeltti. Doğrusu: Bir özelliğin bulunduğu sınıfla korelasyonu yüksekse o özellik o sınıf için kullanılabilir bir özelliktir.

10. Ki-Kare Testi

Filtre yöntemlerinde ikinci yol ki-kare testidir (chi-square test). Ki-kare testi kategorik özellikler için kullanılabilen bir yöntemdir. Burada bir özelliğin o sınıf için uygun olup olmadığına, yani o sınıfla ilişkili olup olmadığına karar verilir.

Bağıntı şöyledir:

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$$

- O_i, gözlenen frekanstır (observed frequency): özelliğin bir değerinin bir sınıfta kaç kez geçtiği.
- E_i, beklenen frekanstır (expected frequency): özellik ile sınıf arasında hiçbir ilişki olmasaydı o hücrede kaç örnek beklenirdi.

Beklenen frekans, hücrenin satır toplamı ile sütun toplamının çarpımının toplam örnek sayısına bölünmesiyle bulunur:

$$E = \frac{\text{satır toplamı} \times \text{sütun toplamı}}{\text{toplam örnek sayısı}}$$

10.1 Örnek: Tenis Oynama Veri Kümesinde Outlook Özelliği

Hoca, karar ağaçlarında görülen tenis oynama (PlayTennis) problemini hatırlattı. 14 gün boyunca hava gözlemleri alınmıştı: rüzgar, sıcaklık, hava durumu (güneşli, bulutlu, yağmurlu), nem. Bu gözlemlere göre o gün tenis oynanıp oynanmadığına karar veriliyordu. Hoca tablonun tamamını tekrar yazmadı, yalnız Outlook (hava durumu) özelliği için ki-kare hesabının nasıl yapıldığını gösterdi.

Outlook özelliğinin üç değeri vardır: Sunny (güneşli), Overcast (bulutlu), Rain (yağmurlu). Bu üç değerin iki sınıftaki dağılımı şöyledir:

Outlook	Yes	No	Toplam
Sunny	2	3	5
Overcast	4	0	4
Rain	3	2	5
Toplam	9	5	14

Hoca tabloyu sözlü olarak okudu: Güneşli olan 5 günün 2'sinde tenis oynanmış, 3'ünde oynanmamış. Bulutlu olan 4 günün 4'ünde de oynanmış. Yağmurlu olan 5 günün 3'ünde oynanmış. Yes'lerin toplamı 9, No'ların toplamı 5'tir.

Adım 1: Beklenen frekanslar. Sunny satırının toplamı 5, Yes sütununun toplamı 9, toplam örnek 14'tür:

- Sunny-Yes: $E = \dfrac{5 \cdot 9}{14} = 3{,}21$
- Sunny-No: $E = \dfrac{5 \cdot 5}{14} = 1{,}79$
- Overcast-Yes: $E = \dfrac{4 \cdot 9}{14} = 2{,}57$
- Overcast-No: $E = \dfrac{4 \cdot 5}{14} = 1{,}43$
- Rain-Yes: $E = \dfrac{5 \cdot 9}{14} = 3{,}21$
- Rain-No: $E = \dfrac{5 \cdot 5}{14} = 1{,}79$

Adım 2: Her hücre için $(O-E)^2/E$. Hoca hesabı şöyle kurdu: Sunny-Yes'te 2 kez oynanmış, beklenen 3,21'dir; $(2-3{,}21)^2/3{,}21$. Sunny-No'da gözlenen 3, beklenen 1,79'dur; $(3-1{,}79)^2/1{,}79$. Overcast için $(4-2{,}57)^2/2{,}57$ ve $(0-1{,}43)^2/1{,}43$. Rain için $(3-3{,}21)^2/3{,}21$ ve $(2-1{,}79)^2/1{,}79$.

Adım 3: Toplam. (ara adım eklendi)

$$\chi^2_{\text{Outlook}} = \frac{(2-3{,}21)^2}{3{,}21} + \frac{(3-1{,}79)^2}{1{,}79} + \frac{(4-2{,}57)^2}{2{,}57} + \frac{(0-1{,}43)^2}{1{,}43} + \frac{(3-3{,}21)^2}{3{,}21} + \frac{(2-1{,}79)^2}{1{,}79}$$

$$\chi^2_{\text{Outlook}} \approx 0{,}459 + 0{,}826 + 0{,}794 + 1{,}429 + 0{,}014 + 0{,}026 \approx 3{,}55$$

Hoca slayttan Outlook'un ki-kare değerinin 3,129, Wind (rüzgar) özelliğinin ki-kare değerinin 3,62 çıktığını okudu. 3,62, 3,129'dan büyük olduğu için "iki özellikten biri seçilecekse Wind seçilmeli" sonucuna vardı. Ki-kare değeri büyük olan özellik sınıfla daha ilişkilidir ve seçilmeye daha uygundur. (Bu sayılar için bkz. Ders Sonrası Notlar.)

Soru: Bir öğrenci 3,21 ve 1,79 frekanslarının nasıl hesaplandığını sordu. **Cevap:** Hoca, gözlenen frekansın özelliğin o sınıfta kaç kez geçtiği (ör. Sunny-Yes için 2), beklenen frekansın da Sunny-Yes hücresinin beklenen değeri olduğunu söyledi. Öğrenci hesabı kendisi yapıp anlayamazsa aradan sonra üzerinden tekrar geçebileceğini ekledi. Öğrenci kısa süre sonra 3,2 değerinin yukarıdaki beklenen değer hesabından geldiğini kendisi buldu.

11. Bilgi Kazancı ve Filtre Yöntemleriyle Adım Adım Eleme

Hoca, ki-kare değerlerinin özellik seçiminde nasıl kullanılacağını şöyle anlattı:

1. Önce bütün özelliklerle model kurulur ve performans ölçülür.
2. Ki-kare değeri en düşük olan özellik çıkarılır, performans yeniden ölçülür. Performans biraz daha yüksek çıkıyorsa o özellik çıkarılabilir.
3. Ki-kare değeri en düşük iki özellik çıkarılıp ölçülür. Performans daha iyiyse iki özellik birden çıkarılabilir.
4. Böyle azalta azalta devam edilir.

Bilgi kazancı (information gain) da aynı şekilde kullanılır. Karar ağacında kullanılan bilgi kazancı da bir filtre yöntemidir. Her özelliğin bilgi kazancı hesaplanır ve bilgi kazancı küçük olan özellikler gözden çıkarılarak model kurulur. Hoca bütün bu filtre yöntemlerinin istatistiksel bilgilere dayandığını vurguladı.

Soru: Bir öğrenci şunu sordu: Diyelim ki bütün özellikler bir şekilde etkili ve hepsini kullandığımızda daha iyi sonuç alıyoruz. Seçim yapmamızın sebebi yalnız performans mı? **Cevap:** Hoca evet dedi ve şöyle açıkladı:

- Seçilen bütün özelliklerin şahane olması gerçek dünyada pek karşılaşılan bir durum değildir. Mutlaka bir ikisi gereksizdir.
- Bazen 20 özellikle başlanır, 3 özelliğe inilir ve çok iyi bir performans elde edilir.
- Özellik sayısı, yani boyut arttıkça bizim gözümüzden bir şeyler kaçabilir. Özellikler arasındaki ilişkiyi (relation) ölçmek imkansızlaşabilir.
- Bu yüzden özellik azaltmanın amacı performansı iyileştirmektir. Model, seçilen özelliklerle kurulur ve performansına bakılır. Sonra Pearson katsayısı, ki-kare ya da bilgi kazancı gibi bir yöntemle etkisinin az olacağı görülen özellikler çıkarılıp başarı yeniden ölçülür. Başarı gerçekten sıçrıyorsa (ör. %70'lerden %80'lere çıkıyorsa) o özellikler kullanılmaz.

  Hocanın vurgusu: Özellik azaltmadaki asıl sebep performansı iyileştirmektir. Bazen 20 özellikle başlayıp 3 özellikle çok iyi sonuç alırsınız.

Hoca burada 10 dakika ara verdi.

12. Sarmalayıcı Yöntemler: İleri Seçim ve Geri Eleme

Aradan sonra hoca özellik seçiminde ikinci yaklaşıma, sarmalayıcı (wrapper) yöntemlere geçti. Bu yaklaşımın çok zahmetli olduğu için pek kullanılmadığını, ama bir yöntem olarak bilinmesi gerektiğini söyledi.

Filtre yöntemlerinde istatistiksel yaklaşımlar (ki-kare, Pearson katsayısı, bilgi kazancı vb.) kullanılıyordu. Önemsiz olduğu düşünülen özellikler çıkarılıyor, öğrenme algoritmasına (ör. k-NN ile bir sınıflandırma) verilip başarı ölçülüyor ve böylece özellikleri çıkarmanın etkisi değerlendiriliyordu. Yani özelliklerin değeri hakkında bir fikir vardı.

Sarmalayıcı yöntemlerde böyle bir mantık yoktur. Özelliklerin değerine ya da önemine bakılmadan özellik eklenir ya da çıkarılır ve her seferinde model başarısı ölçülür. Çıkarmak başarıyı yükseltiyorsa o özellik çıkarılır; eklemek başarıyı artırıyorsa o özellik eklenir. Yani karar, özellik hakkında bir ölçüm yapılmadan, yalnız modelin başarısına göre verilir.

12.1 Ardışık İleri Seçim (Sequential Forward Selection, SFS)

Hiçbir özellik seçilmeden, boş kümeyle başlanır.

1. **Başlangıç:** Seçili özellik kümesi boştur.
2. **Birinci adım:** Eldeki bütün özellikler tek tek denenir. Her biri için yalnız o özellikle model kurulur ve başarı ölçülür. En iyi sonucu veren özellik seçilir. Model şu anda yalnız bu özellikten oluşur.
3. **İkinci adım:** Bu özellik varken, geri kalan özelliklerin her biri tek tek onun yanına eklenerek başarı ölçülür. Böylece iki özellikli modeller oluşur. En başarılı senaryo hangisiyse o ikinci özellik kümeye eklenir.

4. **Tekrar:** Her seferinde bir önceki duruma göre en iyi başarıyı sağlayan özellik kümeye eklenir ve seçili özellik sayısı (k) bir artırılır. Bu işlem başarı artmayana kadar tekrar edilir.

```
S = {}                          (seçili özellikler)
tekrarla:
    her f (S'de olmayan) için: başarı(S + {f}) ölç
    f* = en yüksek başarıyı veren özellik
    eğer başarı(S + {f*}) > başarı(S): S = S + {f*}
    değilse dur
```

12.2 Ardışık Geri Eleme (Sequential Backward Elimination, SBE)

Tahmin edileceği gibi burada tersi yapılır.

1. Önce bütün özellikler varken başarı ölçülür.
2. Her özellik tek tek çıkarılarak başarı ölçülür. Çıkarıldığında en az zarar veren özellik, yani en kötü özellik atılır.
3. Bu işlem başarı değişmeyene kadar tekrar edilir.

12.3 Sarmalayıcı Yöntemlerin Sakıncaları

Hoca iki sakınca üzerinde durdu.

Zaman. 5 özellik olduğunu düşünün. Birinci aşamada en başarılı özellik seçilir; diyelim ki bu 4. özellik oldu. Ardından 4. özellikle birlikte en çok başarı sağlayan ikinci özellik bütün kombinasyonlarla tek tek denenir; diyelim ki 5. özellik çıktı ve o da eklendi. Özellik sayısı çok fazlaysa bu son derece vakit alan bir iştir. Bu yüzden verimli (efektif) değildir.

İlk seçimin bağlayıcılığı. Hoca bunun bütün kapsamlı arama (exhaustive) türü yöntemlerin ortak sıkıntısı olduğunu söyledi. İlk seçilen özellik en yüksek başarıyı sağlayan özelliktir ve diğerleri ona bağlı olarak seçilir. Oysa daha az değerli görünen başka bir özellik seçilseydi, belki başka bir özellik kombinasyonu daha yüksek başarı verecekti. Yani ilk seçilen özelliğe bağlı olarak bazı özellikler hiç kullanılamamış olabilir. Geri elemede de aynı durum vardır: İlk başta elenen bir özellik kalsaydı başka bir kombinasyonla daha iyi sonuç verebilirdi.

13. Plus-L Minus-R Seçimi ve Çift Yönlü Arama

İlk seçimin bağlayıcılığı sorununu biraz hafifletmek için iki yöntem vardır.

13.1 Plus-L Minus-R Seçimi

L ve R, kullanıcının belirlediği değerlerdir. L ile R'nin karşılaştırılması, işe eklemeyle mi yoksa çıkarmayla mı başlanacağını gösterir:

- $L > R$ **ise** ileri seçimle başlanır. Örneğin $L = 3$, $R = 2$ ise önce 3 kez özellik eklenir (ileri seçim), ardından 2 kez özellik çıkarılır (geri eleme). Model başarısı istenen noktaya oturmadıysa yine eklemeye dönülür ve böyle dönerek devam edilir.
- $L < R$ **ise** geri elemeyle başlanır.

Bu sayede şu kontrol edilmiş olur: Eklenen özelliklerin hepsi bir arada gerçekten iyi mi? Üç özellik eklendikten sonra bir tanesi çıkarılıp bakılır; belki tek tek iyi olan özellikler bir araya geldiğinde iyi değildir. Ekleyip çıkararak ilerlenir.

13.2 Çift Yönlü Arama (Bidirectional Search)

Buna çok benzeyen bir yöntemdir. İleri seçim (SFS) ve geri eleme (SBE) paralel olarak çalıştırılır. Biri en iyi özelliği seçip eklerken öbürü kötü özelliği çıkarır. Örneğin 10 özelliğe ulaşıldığında bu 10 özelliğin içinde kötü biri varsa geri eleme onu çıkarır; kalanların içinde iyi bir özellik varsa ileri seçim onu ekler. Böylece yalnız ekleyerek ya da yalnız çıkararak gitmek yerine ekleme ve çıkarma birlikte yapılır.

14. Filtre Yöntemleri Uygulamada Nasıl Kullanılır

Hoca, özellik seçecekseniz uygulamada daha çok ki-kare, korelasyon ve Pearson korelasyon katsayısı gibi filtre yöntemlerinin kullanıldığını söyledi. Bunlar genelde tek başına değil, birlikte kullanılır:

- Pearson korelasyon katsayısına bakılarak bir model oluşturulur.
- Bilgi kazancına bakılarak özellikler belirlenip bir model oluşturulur.
- Ki-kare değerine bakılarak bir model oluşturulur.
- Sonra bu modellerin başarıları birbirine göre değerlendirilir.

Yani "yalnız Pearson'ı seçeyim, özelliklerimi onunla belirleyeyim" denmez. Hangi özelliklerin seçileceğine bu yöntemler hibrit olarak kullanılıp karar verilir. Hoca burada kategorik özellikler için Pearson ve ki-kare kullanıldığını söyledi (bu ifade için bkz. Ders Sonrası Notlar).

15. Özellik Çıkarımı, Özdeğer ve Özvektör

Özellik azaltmanın ikinci yolu çıkarımdır (feature extraction): özellikleri dönüştürerek, daha az boyutlu bir uzaya indirerek kullanmak. Burada da amaç korelasyonlu özelliklerden kurtulmaktır. Korelasyonlu özellikler; korelasyonsuz (uncorrelated), daha az sayıda ve baştakinde var olmayan yeni özelliklere dönüştürülür.

Bunun için en çok kullanılan yöntemlerden biri temel bileşen analizidir (Principal Component Analysis, PCA). Hoca PCA'nın özellik azaltmada en çok kullanılan yöntemlerden biri olduğunu ve özellikle sinyallerle çalışırken hâlâ çok kullanıldığını söyledi.

PCA, özdeğer (eigenvalue) ve özvektör (eigenvector) kavramlarına dayanır. Hoca "eigen" kelimesinin Almanca olduğunu ve "karakteristik" anlamında kullanıldığını söyledi. Amaç, elimizdeki özelliklerden karakteristik, birbiriyle ilişkisiz bilgileri çıkarmaktır: özellikler bir dönüşümden geçtiğinde değeri (yönü) değişmeyen bir alt küme elde etmek.

Tanım. A bir matris, λ bir skaler, x bir vektör olsun. Eğer

$$Ax = \lambda x$$

eşitliği yazılabiliyorsa, λ skalerine A'nın özdeğeri, x vektörüne de özvektörü denir. Amaç, A matrisinin özdeğerlerini ve özvektörlerini bulmaktır. Hoca bunun lineer cebirde görülen bir konu olduğunu hatırlattı.

Çözüm. Eşitlik $Ax - \lambda x = 0$ diye yazılır. x'i parantez dışına almak için birim matris (identity matrix) I kullanılır:

$$(A - \lambda I)\, x = 0$$

Sıfırdan farklı bir x çözümü olması için determinantın sıfır olması gerekir:

$$\det(A - \lambda I) = 0$$

Birim matris, köşegeni 1 ve geri kalanı 0 olan matristir. 2×2 için $I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ olur. Bu durumda çözülmesi gereken şudur:

$$\det \begin{pmatrix} a_{11} - \lambda & a_{12} \\ a_{21} & a_{22} - \lambda \end{pmatrix} = (a_{11} - \lambda)(a_{22} - \lambda) - a_{12}a_{21} = 0$$

15.1 Örnek: 2×2 Matrisin Özdeğer ve Özvektörleri

$$A = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$$

Adım 1: Karakteristik denklem.

$$\det\begin{pmatrix}1-\lambda & 2\\ 2 & 1-\lambda\end{pmatrix} = (1-\lambda)^2 - 4 = 0$$

Adım 2: Özdeğerler. $(1-\lambda)^2 = 4$ çözülür. $1-\lambda = \pm 2$ olduğundan $\lambda_1 = -1$ ve $\lambda_2 = 3$ bulunur.

Adım 3: Özvektörler. Özdeğerler denklemde yerine konarak özvektörler bulunur. (ara adım eklendi)

- $\lambda = 3$ için $(A-3I)x = 0$: $\begin{pmatrix}-2 & 2\\ 2 & -2\end{pmatrix}\begin{pmatrix}x_1\\ x_2\end{pmatrix} = 0$, yani $-2x_1 + 2x_2 = 0$. Özvektör $v = (1,1)$.
- $\lambda = -1$ için $(A+I)x = 0$: $\begin{pmatrix}2 & 2\\ 2 & 2\end{pmatrix}\begin{pmatrix}x_1\\ x_2\end{pmatrix} = 0$, yani $2x_1 + 2x_2 = 0$. Özvektör $v = (1,-1)$.

Hoca bu özvektörlerin ayırt edici bilgiyi gösteren, azaltılmış özellikler olduğunu söyledi.

16. PCA'nın Sezgisi ve Özyüzler

x_1 ve x_2 diye iki özelliği olan örnekleri bir düzlemde düşünün. PCA'nın yapmaya çalıştığı şey, verinin en çok değiştiği yönü bulmaktır. Bu yön birinci temel bileşendir (principal component). Ardından ikinci bileşen gelir ve böyle devam eder. Amaç, bu veriyi temsil edecek temel bileşenleri, yani ana bileşenleri bulmaktır.

Özyüzlerin (eigenfaces) hikayesi. Hoca PCA'nın yüz tanımayla popüler olduğunu anlattı; yanılmıyorsa 1990'lara doğru. O zamana kadar yüz tanıma, dersin başında anlatıldığı gibi yapılıyordu: Önce yüzde gözlerin, burnun, ağzın yeri bulunur, bunlardan üçgenler oluşturulur, bu üçgenlerdeki mesafeler saklanır ve ayırt edici özellik olarak kullanılırdı. Yani ilk çalışmalar mesafe ölçmeye dayanıyordu.

Ama bu, insanın tanıma mantığına aykırıdır. İnsanlar birbirlerini böyle tanımaz; yüz tanırken kimse kimsenin iki gözü arasındaki mesafeyi aklında tutmaz. Genel bilgiye, yüzün bütününe bakılır.

Hoca burada Matthew Turk ve Alex Pentland'ın çok meşhur çalışmasını anlattı. Bu makalenin, yanılmıyorsa, 20. yüzyılda çalışmaların yönünü değiştiren ilk 100 makale arasında seçildiğini söyledi. Turk ve Pentland "insanlar yüzün tamamını değerlendiriyor, o zaman biz de yüzün tamamını değerlendirelim" diyor.

Peki yüzün tamamı nasıl değerlendirilecek? Burada bir sorun vardır: Yüzün üzerindeki her pikseli bir özellik olarak kullanmaya kalkarsanız, piksellerin çoğu yanak ya da alın gibi herkeste ortak olan bölgelere aittir. Bu ortak bilgi yüzünden ayırt ediciliği yakalamak çok zordur. O halde soru şudur: Yüz piksellerinin tamamından karakteristik bilgi çıkarılabilir mi?

Turk ve Pentland çok yaratıcı bir yol buluyor. Tanınmak istenen beş kişi olsun. Bu kişilerin yüz fotoğraflarının birbirlerine göre değişimine bakılır; bu değişim kovaryanstır. Kovaryans matrisinden özdeğerler ve özvektörler çıkarılır ve bunlar özellik olarak kullanılır. Böylece insanların yüzlerindeki karakteristik bilgi yakalanmış ve kullanılmış olur. Hoca, bu çalışmadan sonra bu yaklaşımın yaygın olarak kullanıldığını söyledi.

17. PCA ile Yüz Tanıma: Eğitim ve Test Adımları

Hoca önce kovaryansın birden fazla değişkene nasıl genişlediğini hatırlattı. İki özelliğin birbirine göre değişimi $\mathrm{cov}(X,Y)$ ile bulunuyordu. Üç bilgi olsaydı, bunların birbirlerine göre değişimine bir kovaryans matrisi kurarak bakılırdı: X'in X'e göre değişimi, X'in Y'ye göre değişimi, X'in Z'ye göre değişimi, Y'nin X'e göre değişimi ve böyle devam eder. Her biri tek tek hesaplanır. Yüz tanımadaki mantık da buna dayanır: Bütün yüzlerin birbirlerine göre değişimine, yani kovaryansına bakılır. Kovaryans matrisinden temel bileşenler çıkarılınca özellikler bulunmuş olur.

17.1 Eğitim Aşaması

Eğitim aşaması, temel bileşenlerin elde edildiği aşamadır. Elinizde $N \times N$ boyutlu yüz görüntüleri ve tanınmak istenen M kişi olsun.

1. **Vektöre çevirme:** Her görüntü matristen tek boyutlu bir vektöre çevrilir. Birinci kişiye A, ikinci kişiye B diyelim (ör. Ali, Banu, Hüseyin). $N \times N$ görüntü, $N^2 \times 1$ boyutlu bir vektör olur. Hoca, eldeki veri banka işlemleri gibi zaten tek boyutlu olsaydı bu adıma gerek olmayacağını söyledi. Yüz tanımayı yalnız görsel olduğu ve akılda kolay kaldığı için örnek olarak seçtiğini belirtti; PCA'nın en çok kullanıldığı yerlerden biri de burasıdır.
2. **Ortalama yüz:** Kovaryans hesabında ortalama çıkarılması gerektiği için önce ortalama yüz hesaplanır:
$$\Psi = \frac{1}{M} \sum_{i=1}^{M} \Gamma_i$$
3. **Ortalamayı çıkarma:** Ortalama yüz bütün görüntülerden çıkarılır: $\Phi_i = \Gamma_i - \Psi$. Buna ortalaması çıkarılmış (mean-centered) veri denir.
4. **Matrise yerleştirme:** Ortalaması çıkarılmış yüzler yan yana konarak bir matris kurulur: $A = [\Phi_1 \; \Phi_2 \; \cdots \; \Phi_M]$.
5. **Kovaryans matrisi:** Bu matris transpozuyla çarpılınca herkesin birbirine göre durumunu gösteren kovaryans matrisi elde edilir: $C = AA^T$. Hoca "olay bundan ibaret" dedi.
6. **Özdeğer ve özvektörler:** Kovaryans matrisinin özdeğerleri ve özvektörleri, az önceki örnekte görüldüğü gibi bulunur.
7. **Seçim:** Sıfırdan farklı özdeğerler büyükten küçüğe sıralanır ve en büyüklerden belli bir yüzdesi alınır (ör. %70'i). Bunlara karşılık gelen özvektörler seçilir.
8. **Özuzaya geçirme:** Ortalaması çıkarılmış her görüntü, seçilen özvektörlerle çarpılarak özuzaya (eigenspace) geçirilir: $\Omega_i = V^T \Phi_i$. Artık özellikler baştaki piksel değerleri değil, bu yeni değerlerdir.

17.2 Test Aşaması

Bilinmeyen bir Y örneği geldiğinde aynı işlemler ona da uygulanır:

1. Y tek boyutlu bir vektöre çevrilir.
2. Eğitimde hesaplanan ortalama yüz çıkarılır (mean-centered hale getirilir).
3. Seçilen özvektörlerle çarpılarak özuzaya geçirilir.
4. Elde edilen özellikler öğrenme modeline verilir. k-en yakın komşu kullanılıyorsa k-NN'e, destek vektör makinesi kullanılıyorsa SVM'e verilir.

Hoca, başlangıçtaki bilginin görüntü olması gerekmediğini vurguladı. Veri ne olursa olsun, indirgendikten sonra bu yeni özellikler kullanılabilir.

18. Örnek: 3×3 Görüntülerle PCA

Hoca hazırladığı bir örneği ekranda paylaştı. Dört sınıf vardı: X_1, X_2, X_3, X_4. Her biri 3×3 piksellik gri seviye bir görüntüydü. Y ise sınıfı bilinmeyen örnekti. Amaç, önce özdeğer ve özvektörlerle özellikleri azaltmak, ardından tanıma yapmaktı.

Piksel renkleri 0 ile 255 arasında değişiyordu: 0 siyahı, 255 beyazı gösteriyordu. Örneğin X_1'in bir satırı 225, 229, 48 idi. 225 ve 229 beyaza yakın, 48 siyaha yakın pikselleri gösteriyordu.

Adımlar şöyle ilerledi:

1. **Vektöre çevirme:** 3×3 matrisler 9×1 vektörlere çevrildi.
2. **Ortalama:** Bu vektörlerin ortalaması bulundu. Her satırın (pikselin) ortalaması ayrı hesaplandı.
3. **Ortalamayı çıkarma:** Ortalama her vektörden çıkarıldı. Örneğin bir pikselde $225 - 171{,}5 = 53{,}5$ elde edildi. Böylece bütün örnekler mean-centered hale getirildi.
4. **Matris:** Ortalaması çıkarılmış dört vektör yan yana konarak 9×4 boyutlu bir matris elde edildi: 9 özellik (9 piksel), 4 örnek.
5. **Kovaryans:** 9×4 matris 4×9 transpozuyla çarpılınca 9×9 boyutlu kovaryans matrisi elde edildi.

6. **Özdeğerler:** Kovaryans matrisinin sıfırdan farklı 3 özdeğeri çıktı: λ_1, λ_2, λ_3. Bunlar büyükten küçüğe dizildi.
7. **Özvektörler:** Bu özdeğerlere karşılık gelen özvektörler yeni özellikleri verdi. Her özvektör 9 satırlık olduğundan ve 3 özvektör seçildiğinden 9×3 boyutlu bir özvektör matrisi oluştu.
8. **Özuzaya geçirme:** Ortalaması çıkarılmış her görüntü bu özvektörlerle çarpıldı. Özvektör matrisinin transpozu 3×9, ortalaması çıkarılmış görüntü 9×1 olduğu için sonuç 3×1 oldu. Yani 9 piksel (9 özellik) 3 özelliğe inmiş oldu.
9. **Test:** Y görüntüsü önce 9×1 vektöre çevrildi, ortalaması çıkarıldı ve özvektörlerle çarpılarak özuzaya geçirildi.
10. **Tanıma:** Bu örnekte sınıflandırma için yalnız Öklid mesafesi kullanıldı. Y'nin özuzaydaki bütün örneklere mesafesi ölçüldü ve en düşük mesafe X_2'ye çıktı. Hoca görüntülere bakınca Y'nin gerçekten en çok X_2'ye benzediğini gösterdi.

19. Kaç Özvektör Seçilmeli

Soru: Bir öğrenci, 2×2 örnekte her vektör için iki özdeğer çıktığını hatırlatıp "en büyük olanı mı seçiyoruz?" diye sordu. **Cevap:** Hoca hayır dedi. Kaç kök varsa o kadar özdeğer çıkar. 2×2 örnekte iki kök vardı, bu yüzden iki özdeğer ve bunlara karşılık gelen iki özvektör bulundu. Kovaryans matrisinde de kaç kök varsa o kadar özdeğer çıkar. Bunların içinden sıfırdan büyük, yani pozitif olan özdeğerlere karşılık gelen özvektörler seçilir.

Hoca seçimin nasıl yapılacağını yine iki boyutlu nokta çizimiyle anlattı. En büyük özdeğere karşılık gelen özvektör, en değerli temel bileşeni gösterir. Noktaları en iyi temsil eden yön, en karakteristik temel bileşendir. İkinci yön ikinci değerli bileşendir. Böyle devam edilebilir, ama sonrakiler artık o kadar değerli değildir. Özdeğerin değeri düştükçe ona karşılık gelen özvektörün değeri de düşer.

> **Hocanın vurgusu:** Pozitif olan bütün özdeğerlere karşılık gelen özvektörleri seçmek doğru değildir. Kaç tane seçileceği deneysel olarak, modelin başarısına göre belirlenir. Genellikle ilk %60-70'ini seçmek doğrudur.

Soru: Bir öğrenci negatif özdeğerlerin neden kullanılmadığını, neden yalnız pozitiflerin seçildiğini sordu. **Cevap:** Hoca, pozitif ve büyük özdeğerlerin anlam ifade ettiği için seçildiğini söyledi. Negatif bir özdeğere karşılık gelen özelliği uygulayıp ne olduğuna bugüne kadar hiç bakmadığını, aslında bakılıp sonucun görülebileceğini ekledi. Özdeğerler yine büyükten küçüğe dizilir. (Bu soru için bkz. Ders Sonrası Notlar.)

Özyüz görselleri. Hoca ardından gerçek bir yüz veri kümesinden elde edilmiş görselleri gösterdi:

- **Ortalama yüz:** Gerçekten bir insan yüzüne benziyordu. Benzerliklerin fazla olduğu yerler, örneğin göz bölgesi, daha belirgindi. Dudak bölgesinde ise insanların dudaklarının farklı boyutta olması yüzünden belirginlik biraz bozulmuştu.
- **Özvektörler (özyüzler):** Birinci özvektör insan yüzüne daha çok benziyordu. İkincide bozulmalar başlıyordu. Üçüncü ve dördüncü fena değildi; dikkat edilirse dördüncüde sakallı kişilerden kaynaklanan bir karakteristik ortaya çıkıyordu. Sonrakiler yavaş yavaş bozuluyordu.

Buradan hocanın vardığı sonuç şuydu: Seçilen özvektör sayısı arttıkça karakteristik olmayan özellikler de seçilmeye başlanır. Bu yüzden model seçiminde olduğu gibi "5 özvektör seçtim, başarıyı ölçtüm; 10 seçtim, başarıyı ölçtüm" diye denenerek karar verilir.

Hoca son olarak bir kişinin görüntüsünün 8 özvektörle özuzaya geçirilmesini gösterdi. Bu 8 özvektörle elde edilen sayısal değerler görüntüye dönüştürüldüğünde kişinin yüzüne benzeyen bir görüntü elde ediliyordu. Böylece başlangıçtaki özelliklerden gereksiz olanlar elenmiş olur.

20. PCA'nın Kullanım Alanları

Hoca PCA'nın kullanım alanlarını şöyle özetledi:

- **Görüntü işleme:** Kullanımı yüz tanımayla başladı.

- **Ses sinyalleri:** Daha sonra ses sinyallerinde, sinyalin karakteristiklerini çıkarıp boyutunu azaltmak için kullanıldı. Hoca eskiden sinyal işlemede çok kullanıldığını söyledi.
- **Düz (tablo) veriler:** Bugün banka verisi gibi düz verilerde de çok karşınıza çıkar.

Gizlilik avantajı. Hoca PCA'nın bir avantajına daha dikkat çekti: Veri dönüştürüldüğü için orijinal veri kaybolur. Banka verilerinde güvenlik açısından bir endişe vardır. Paylaşılan verilerde kimlik bilgileri görünmez, ama çok istenirse yine de tersine mühendislik (reverse engineering) yapılıp kartın sahibi bulunabilir. Örneğin kişinin gittiği ülkeler ve alışveriş yaptığı yerler gibi açık bilgilerden geriye doğru giderek kart sahibine ulaşılabilir. Bu yüzden kamuya açık (public) paylaşılan bu tür verilerde dikkat edilirse veri PCA ile dönüştürülmüş olarak paylaşılır. Böylece hem karakteristik özellikler çıkarılır hem de orijinal özellikler gizlenir. Hoca bunun şu anda çok yapılan bir şey olduğunu söyledi.

21. Otokodlayıcı ve Derin Öğrenmede Özellik Seçimi

Hoca sınıfa PCA'dan daha başarılı bir özellik çıkarma (dönüştürme) yöntemi bilen olup olmadığını sordu.

Soru: Bir öğrenci t-SNE olabilir mi diye önerdi. Hoca açılımını sordu. Öğrenci, PCA ile t-SNE'yi bir görselleştirme aracında 3 boyutta görselleştirme yaparken gördüğünü ve oradan aklında kaldığını söyledi, açılımını bilmiyordu. Hoca da t-SNE'nin açılımını bilmediğini söyledi. Bir başka öğrenci CNN (evrişimli sinir ağı) dedi. Hoca evrişimli sinir ağının kendi başına bir özellik dönüştürme yöntemi olmadığını, buna birazdan değineceğini söyledi.

Bu konuşma üzerine hoca lisansüstü öğrencilere bir uyarıda bulundu:

> **Hocanın vurgusu:** Lisansüstü öğrencisiniz; yöntemleri gerçek isimleriyle bilin. Araçlar (tool) üzerinden konuşmayın, araçlarda geçen adları kullanmayın. Örneğin WEKA'da destek vektör makinesine SVM denmez, SMO denir ve bu başka bir şeyin kısaltmasıdır. Aracın içinde nasıl geçtiğini bilmek de önemlidir, ama yöntemin adını bilmeniz gerekir.

Hoca cevabı kendisi verdi: PCA gibi özellikleri dönüştürerek kullanmak için şu anda en çok kullanılan ve PCA'dan daha başarılı olan yaklaşım otokodlayıcıdır (autoencoder).

Klasik makine öğrenmesi ile derin öğrenme. Hoca, CNN diyen öğrencinin de aslında fena bir şey söylemediğini belirtti. Derin öğrenmenin pek çok klasik yöntemden daha başarılı olmasının sebebi buradadır. Klasik makine öğrenmesinde akış şöyledir:

```
örnekler --> ön işleme --> özellik seçimi / çıkarımı --> model seçimi (k-NN, karar ağacı...)
```

Bu akışta başarı her adıma göre değişir. Özellikleri elle seçerseniz başarı bir olur. Filtre yöntemlerinden biriyle seçerseniz başka olur. Sarmalayıcıyla seçerseniz başka olur. Çıkarım yaparsanız yine başka olur. Hoca bunların ne kadar zahmetli işler olduğunu az önce anlattığını hatırlattı.

Derin öğrenmede ise özellik seçimi de modelin içine alınır. Hoca bunu sevdiği bir benzetmeyle anlattı: Bir taraftan ineği verirsiniz, öbür taraftan hamburger çıkar. Derin öğrenme "o işi bana bırak, sen ver, ben özellikleri senin için seçerim" der. Evrişimli sinir ağında (Convolutional Neural Network, CNN) görüntüye uygulanan evrişim (convolution) işlemindeki filtreler, en uygun özellikleri sistemin içinde seçer.

Otokodlayıcı. Otokodlayıcı aslında eğitmensiz (unsupervised) bir öğrenme yöntemidir. Ama özellik çıkarma aşamasında kesilirse çok iyi özellikler verir. Derin öğrenmede de bu çok yapılır: Özellik çıkarma yapıldıktan sonra, derin öğrenme modeline geçmeden ağ kesilir ve bu özellikler kullanılarak klasik makine öğrenmesi yöntemleriyle (ör. destek vektör makinesi) sınıflandırma yapılır.

22. Gömülü Yöntemler

Hoca ders sonunda, özellik seçimindeki üçüncü yaklaşımı, gömülü (embedded) yöntemleri anlatmayı unuttuğunu fark etti ve "önem vermediğim için olsa gerek" diyerek kısaca anlattı.

Gömülü yöntemde özellikler öğrenme yönteminin kendi içinde seçilir. Hoca sınıfa, iyi bildikleri ve içinde özellik seçimi yapılan bir öğrenme yöntemi sordu.

Soru: Hoca, içinde özelliklerin seçildiği bir öğrenme yöntemi adı istedi. **Cevap:** Bir öğrenci karar ağaçları dedi. Hoca "kesinlikle" dedi ve açıkladı: Karar ağacında en iyi özellik köke konur, sonra aşağı doğru inilir. Bütün örnekler sınıflandırılabildiği noktada ağaç kesilir. Kesilen noktada belki de bütün özellikler düğümlerde yer almamıştır. Yani bütün özellikler kullanılmamıştır. Bütün özelliklerin seçilmemesi, özellik seçimi yapıldığı anlamına gelir. Bu yüzden karar ağacı gömülü bir yöntemdir.

Hoca rastgele ormanın (random forest) da gömülü bir yöntem olduğunu, derste anlatmadığını ama anlatılabileceğini ekledi.

Dersin Toparlaması

Bu derste bir öğrenme modelinde hangi özelliklerin kullanılacağının nasıl belirleneceği anlatıldı. Hedef her zaman aynıdır: sınıf içi benzerliği çok, sınıflar arası benzerliği az yapan, gerçekten ayırt edici özellikleri bulmak. Özellikleri hissiyatla seçmek uzun yıllar yapıldı, ama matematiksel bir değerlendirme gerekir. Kategorik özellikler bazı yöntemlerde doğrudan kullanılabilir ya da tamsayı veya one-hot kodlamayla sayıya çevrilebilir. One-hot kodlama, hocanın sahtekarlık projesinde olduğu gibi 40 özelliği 4000'e çıkarabilir; bu da hem hızı hem başarıyı olumsuz etkiler.

Özellik azaltmanın iki yolu vardır. Özellik seçiminde özellikler dönüştürülmeden elenir. Filtre yöntemleri (Pearson korelasyon katsayısı, ki-kare, bilgi kazancı) istatistiğe dayanır. Sarmalayıcı yöntemler (ileri seçim, geri eleme, plus-L minus-R, çift yönlü arama) özelliğin önemine bakmadan ekleyip çıkararak model başarısını ölçer; zahmetli oldukları için az kullanılırlar. Gömülü yöntemlerde (karar ağacı, rastgele orman) seçim yöntemin içinde yapılır. Özellik çıkarımında ise özellikler yeni, korelasyonsuz ve daha az sayıda özelliğe dönüştürülür. Bunun en bilinen yöntemi PCA'dır: veriler ortalaması çıkarılıp kovaryans matrisine dönüştürülür, en büyük özdeğerlere karşılık gelen özvektörler seçilir ve veri bu özuzaya izdüşürülür. Turk ve Pentland'ın özyüzleri bu fikrin yüz tanımadaki ünlü uygulamasıdır. Bugün PCA'dan daha başarılı olan yaklaşım otokodlayıcıdır; derin öğrenme ise özellik seçimini modelin içine alır.

Hoca bir sonraki derste işbirlikçi filtrelemeyi (collaborative filtering) anlatacağını ve derin öğrenme kavramlarına çok basit bir giriş yapacağını söyledi.

Ders Sonrası Notlar

Ders sonrası not: Hoca slayttan Outlook için ki-kare değerini 3,129, Wind için 3,62 olarak okudu ve Wind'in seçilmesi gerektiğini söyledi. Derste verilen Outlook tablosuyla (2/3, 4/0, 3/2) hesaplanınca $\chi^2_{\text{Outlook}} \approx 3{,}55$ çıkıyor, 3,129 değil. Standart tenis veri kümesinde Wind özelliği (Weak: 6 Yes / 2 No, Strong: 3 Yes / 3 No) için $\chi^2_{\text{Wind}} \approx 0{,}93$ çıkıyor. Yani bu veriyle Outlook sınıfla daha ilişkilidir ve seçilmesi gereken Outlook olur. Slayttaki Wind tablosu görülemediği için 3,62 değerinin kaynağı doğrulanamadı.

Ek bilgi (derste anlatılmadı): Ki-kare değerleri, serbestlik dereceleri farklı özellikler arasında doğrudan karşılaştırılamaz. Serbestlik derecesi $(\text{satır sayısı} - 1)(\text{sütun sayısı} - 1)$ ile bulunur: Outlook için $(3-1)(2-1) = 2$, Wind için $(2-1)(2-1) = 1$. Değer sayısı çok olan özelliğin ki-kare değeri doğal olarak büyük çıkmaya eğilimlidir. Bu yüzden uygulamada ham ki-kare değerleri yerine bunlara karşılık gelen p-değerleri karşılaştırılır.

Ders sonrası not: Hoca 01:18:22 civarında "kategorik özelliklerimiz varsa Pearson kullanılıyor" dedi. Doğrusu tersidir: Pearson korelasyon katsayısı sayısal özellikler arasındaki doğrusal ilişkiyi ölçer. Kategorik özellikler için ki-kare testi kullanılır. Hoca da dersin başında ki-karenin kategorik özellikler için olduğunu söylemişti.

Ders sonrası not: Bir öğrenci neden yalnız pozitif özdeğerlerin seçildiğini sordu; hoca negatif özdeğerleri hiç denemediğini söyledi. Kovaryans matrisi pozitif yarı tanımlı (positive semi-definite) bir matristir, bu yüzden özdeğerleri hiçbir zaman negatif olmaz; yalnız sıfır ya da pozitif çıkar. Seçimde asıl ayrım, sıfır olan ve sıfırdan farklı özdeğerler arasındadır. Derste 2×2 örnekte görülen $\lambda = -1$, matris bir kovaryans matrisi olmadığı için çıkabiliyor.

Ek bilgi (derste anlatılmadı): Örnekte 4 görüntüden sıfırdan farklı yalnız 3 özdeğer çıkması tesadüf değildir. Ortalama çıkarıldıktan sonra M örnekten kurulan kovaryans matrisinin rankı en fazla $M-1$'dir; çünkü ortalaması çıkarılmış vektörlerin toplamı sıfırdır ve biri diğerlerinden yazılabilir. Ayrıca $C = AA^T$ ifadesinde normalde $\frac{1}{M-1}$ çarpanı da bulunur; bu çarpan özvektörleri değiştirmez, yalnız özdeğerleri ölçekler.

Ders sonrası not: 2×2 örnekte altyazıda iki özvektör de "1, 1" gibi okunuyor. Doğrusu $\lambda = 3$ için $(1, 1)$, $\lambda = -1$ için $(1, -1)$'dir (python ile doğrulandı).

Ders sonrası not: Hoca "eigen" kelimesinin Almanca'da "karakteristik" anlamına geldiğini söyledi. Kelimenin asıl anlamı "kendi, kendine özgü"dür; Türkçe karşılığı "öz" bu yüzden seçilmiştir (özdeğer, özvektör). "Kendine özgü" anlamı, hocanın "karakteristik" yorumuyla örtüşür.

Ders sonrası not: t-SNE'nin açılımı t-distributed Stochastic Neighbor Embedding'dir (van der Maaten ve Hinton, 2008). PCA'dan farklı olarak doğrusal olmayan bir yöntemdir ve daha çok yüksek boyutlu veriyi 2-3 boyutta görselleştirmek için kullanılır. WEKA'daki SMO ise Sequential Minimal Optimization'ın kısaltmasıdır; SVM'yi eğitmek için kullanılan bir optimizasyon algoritmasıdır.

Ders sonrası not: Özyüz makalesi: M. Turk ve A. Pentland, "Eigenfaces for Recognition", Journal of Cognitive Neuroscience, 1991. Gömülü yöntemlere hocanın verdiği karar ağacı ve rastgele orman örneklerinin yanında L1 düzenlileştirmeli (Lasso) doğrusal modeller de yaygın bir örnektir.

Ders sonrası not: Altyazıda bozuk geçen terimlerin doğru yazımları: "Füçür" = feature, "Nayibayez" = Naive Bayes, "İntegre Renk Koding" = integer encoding, "Huvan Hatay İnkoding / OneHot" = one-hot encoding, "having distance / Heming distance" = Hamming distance, "öklük distance / öklit distance" = Euclidean (Öklid) distance, "papyrus'in korelasyon" = Pearson korelasyonu, "key square / key-sucur" = chi-square (ki-kare), "A-Book / alt book" = Outlook, "reyni" = rainy, "orkest" = overcast, "Information Game" = information gain, "egzostif" = exhaustive, "Aygın / aygün" = eigen, "principal komponent analiz" = principal component analysis, "o tenkodur" = autoencoder, "communication on neural network" = convolutional neural network, "VEKA" = WEKA.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.